Bit Manipulation

1) Logical And(&)

a	&	b	
1		0	0
0		1	0
0		0	0
1		1	1

BTW
↳ (n & 1) → Tell if a number is odd or even.

2) Logical or(|)

a	|	b	
0		0	0
1		0	1
0		1	1
1		1	1

3) Logical not (~)

a	~a
1	0
0	1

4) Logical xor (^)

a	^	b	
0		0	0
1		0	1
0		1	1
1		1	0

Properties of XOR

1) Identity $x \wedge 0 = x$

2) Self Inverse $x \wedge x = 0$

3) Commutative $x \wedge y = y \wedge x$

4) Associative $(x \wedge y) \wedge z = x \wedge (y \wedge z)$

5) Shift operator

a) Left Shift (<<)

$0001 << 1 \rightsquigarrow 0010$ ↷ 2

$0001 << 2 \rightsquigarrow 0100$ ↷ 4

↳ multiplying with 2^i

b) Right Shift(>>)

$00001000 >> 1 \rightsquigarrow 00000100$

$00001000 >> 2 \rightsquigarrow 00000010$

↳ dividing by 2^i

Common Bit tasks

```
bool isOdd(int n){
    return (n & 1)
}
```

(n & 1) → 2^0

```
int getBit(int &n, int i){
    int mask = (1<<i);  ⇝ 00000001 << i
    int bit  = (n & mask)>0 ? 1 : 0;
    return bit;
}

int setBit(int &n, int i){
    int mask = (1<<i);
    int ans = n | mask;
    return ans;
}
```

```
11101111
00010000
⟶
11111111
```

```
void clearBit(int &n, int i){
    int mask = ~(1<<i);  → 1 → 00000001
    n = n & mask;          ~1 → 11111110
}

void updateBits(int &n, int i, int v){
    int mask = ~(1<<i);
    int cleared_n = n & mask;
    n = cleared_n | (v<<i);
}
```

```
int clearLastIBits(int n, int i){
    int mask = (-1 << i);
    return n & mask;
}
```

-1 ⇝ 1111 1111 (Two's Compliment)

```
int clearRangeItoJBits(int n, int i, int j){
    int ones = ~0;
    int a = ones << (j+1);
    int b = (1<<1)-1;
    int mask = a|b;
    int ans = n & mask;
    return ans;
}
```

⟶

Ques → Find Unique Number.

{ 2 2, 3, 3, 3